

Guía de Mapeo JPA para Proyecto Cine

1. Explicación de las Anotaciones JPA Utilizadas

Antes de pasar al código, es fundamental entender qué hace cada anotación para justificar las decisiones tomadas:

- **@Entity**: Indica a Spring/Hibernate que esta clase representa una tabla en la Base de Datos.
 - **@Table(name = "...")**: Define el nombre exacto de la tabla en SQL.
 - **@Id y @GeneratedValue**: Definen la Clave Primaria (PK) y cómo se autogenera (usualmente **IDENTITY** para autoincremental).
 - **@OneToOne (Uno a Muchos)**:
 - Se usa en el lado "Uno" de la relación (ej. Director).
 - **Importante**: Casi siempre lleva **mappedBy**. Esto indica que esta entidad **NO** tiene la columna de clave foránea (FK) en su tabla. Le dice a Hibernate: "Ve a la otra clase y busca el atributo llamado X para saber cómo relacionarnos".
 - **@ManyToOne (Muchos a Uno)**:
 - Se usa en el lado "Muchos" (ej. Película, Crítica).
 - **Aquí es donde reside físicamente la Clave Foránea (FK)**.
 - Se acompaña de **@JoinColumn(name = "...")** para nombrar esa columna FK en la tabla.
 - **@ManyToMany (Muchos a Muchos)**:
 - Requiere una **tabla intermedia**.
 - Un lado debe ser el "Dueño" (Owner) que lleva **@JoinTable** (define el nombre de la tabla intermedia y las columnas FK).
 - El otro lado es el "Inverso" y lleva **mappedBy**.
-

2. Implementación de las Entidades

He modificado tus clases para reflejar fielmente el diagrama E-R proporcionado.

A. Entidad: PELICULA (Entidad Central)

Esta es la entidad más compleja porque conecta con casi todo.

Decisiones de diseño:

1. **Categoría, Idioma, Plataforma (N:M)**: He decidido que **Película** sea la entidad "propietaria" (Owner) de estas relaciones. Esto significa que al guardar una Película,

puedes guardar sus categorías/idiomas directamente. Es lo más lógico en un formulario de creación de películas.

2. **Director (N:1)**: La película tiene la FK del director.
3. **Actor (N:M)**: En tu código original, **Actor** era el propietario. Lo he mantenido así (con **mappedBy** aquí), aunque funcionalmente es bidireccional.
4. **Crítica (1:N)**: Una película tiene una lista de críticas.

Java

```
package com.dam2.Practica1.models;
```

```
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
```

```
@Entity
@Table(name = "peliculas")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Pelicula {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, length = 120)
    private String titulo;
```

```
    private int duracion;
```

```
    @Column(name = "fecha_estreno")
    private LocalDate fechaEstreno;
```

```
    @Column(columnDefinition = "TEXT") // Permite textos largos para sinopsis
    private String sinopsis;
```

```
    private int valoracion;
```

```
// =====
// RELACIONES MANY-TO-ONE (Lado dueño de la FK)
// =====
```

```
/**
 * RELACIÓN N:1 con Director.
```

```

    * La tabla 'películas' tendrá una columna 'director_id'.
    */
    @ManyToOne(fetch = FetchType.EAGER) // EAGER: Carga el director automáticamente
    al pedir la peli
    @JoinColumn(name = "director_id", nullable = true)
    @JsonIgnoreProperties("películas") // Evita bucle infinito JSON
    private Director director;

    // =====
    // RELACIONES ONE-TO-MANY (Lado inverso)
    // =====

    /**
     * RELACIÓN 1:N con Crítica.
     * mappedBy = "película": Indica que la FK está en la clase Critica, atributo 'película'.
     */
    @OneToMany(mappedBy = "película", cascade = CascadeType.ALL, orphanRemoval =
true)
    @JsonIgnoreProperties("película")
    private List<Critica> criticas = new ArrayList<>();

    // =====
    // RELACIONES MANY-TO-MANY
    // =====

    /**
     * RELACIÓN N:M con Actor.
     * mappedBy = "películas": Indica que Actor define el @JoinTable.
     * Aquí solo reflejamos la relación.
     */
    @ManyToMany(mappedBy = "películas")
    @JsonIgnoreProperties("películas")
    private List<Actor> actors = new ArrayList<>();

    /**
     * RELACIÓN N:M con Categoría.
     * Película actúa como DUEÑO de la relación.
     * Crea tabla intermedia 'película_categoria'.
     */
    @ManyToMany
    @JoinTable(
        name = "película_categoria",
        joinColumns = @JoinColumn(name = "película_id"),
        inverseJoinColumns = @JoinColumn(name = "categoria_id")
    )
    @JsonIgnoreProperties("películas")
    private List<Categoría> categorias = new ArrayList<>();

```

```

/**
 * RELACIÓN N:M con Idioma.
 * Crea tabla intermedia 'pelicula_idioma'.
 */
@ManyToMany
@JoinTable(
    name = "pelicula_idioma",
    joinColumns = @JoinColumn(name = "pelicula_id"),
    inverseJoinColumns = @JoinColumn(name = "idioma_id")
)
@JsonIgnoreProperties("peliculas")
private List<Idioma> idiomas = new ArrayList<>();

/**
 * RELACIÓN N:M con Plataforma.
 * Crea tabla intermedia 'pelicula_plataforma'.
 */
@ManyToMany
@JoinTable(
    name = "pelicula_plataforma",
    joinColumns = @JoinColumn(name = "pelicula_id"),
    inverseJoinColumns = @JoinColumn(name = "plataforma_id")
)
@JsonIgnoreProperties("peliculas")
private List<Plataforma> plataformas = new ArrayList<>();
}

```

B. Entidad: CRITICA

Esta entidad conecta Usuarios con Películas.

Decisión clave: Al ser el lado "Muchos" de ambas relaciones (1 Usuario escribe N críticas, 1 Película recibe N críticas), esta tabla **debe contener las Claves Foráneas**.

Java

```
package com.dam2.Practica1.models;
```

```
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;
```

```
@Entity
@Table(name = "criticas")
@Data
@AllArgsConstructor
```

```

@NoArgsConstructor
public class Critica {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "TEXT")
    private String comentario;

    @Column(nullable = false)
    private int nota;

    @Column(nullable = false)
    private LocalDate fecha; // Cambiado a LocalDate, mejor que String para fechas

    // =====
    // CLAVES FORÁNEAS (Dueño de la relación)
    // =====

    /**
     * Relación N:1 con Película.
     * Muchos críticas pertenecen a una película.
     * Se crea columna 'pelicula_id'.
     */
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "pelicula_id", nullable = false)
    @JsonIgnoreProperties("criticas") // Al serializar critica, no muestres toda la lista de
criticas de la peli
    private Pelicula pelicula;

    /**
     * Relación N:1 con Usuario.
     * Muchas críticas son escritas por un usuario.
     * Se crea columna 'usuario_id'.
     */
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "usuario_id", nullable = false)
    @JsonIgnoreProperties("criticas")
    private Usuario usuario;
}

```

C. Entidad: USUARIO

El usuario escribe críticas.

```

Java
package com.dam2.Practica1.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name="usuarios")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120, unique = true)
    private String userName;

    @Column(nullable = false, length = 120)
    private String email;

    @Column(nullable = false, length = 120)
    private String password;

    @Column(nullable = false, length = 120)
    private String rol;

    /**
     * Relación 1:N con Crítica.
     * Un usuario tiene muchas críticas.
     * mappedBy apunta al campo 'usuario' en la clase Critica.
     */
    @OneToMany(mappedBy = "usuario", cascade = CascadeType.ALL)
    @JsonIgnoreProperties("usuario") // Evita que al pedir usuario, traiga criticas, que traigan
    usuario...
    private List<Critica> criticas = new ArrayList<>();
}

```

D. Entidad: ACTOR

Mantenemos tu lógica original donde Actor es dueño de la relación con Película.

```

Java
package com.dam2.Practica1.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "actores")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Actor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    /**
     * Relación N:M con Pelicula.
     * ACTOR es el propietario (Owner) porque tiene @JoinTable.
     * Crea la tabla 'actor_pelicula'.
     */
    @ManyToMany
    @JoinTable(
        name="actor_pelicula",
        joinColumns = @JoinColumn(name="actor_id"),
        inverseJoinColumns = @JoinColumn(name="pelicula_id"))
    @JsonIgnoreProperties("actors") // Evita recursión
    private List<Pelicula> peliculas = new ArrayList<>();

    // Método helper para consistencia bidireccional
    public void addPelicula(Pelicula p){
        this.peliculas.add(p);
        p.getActors().add(this);
    }
}

```

E. Entidad: DIRECTOR

Lado "Uno" de la relación con Película.

```

Java
package com.dam2.Practica1.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.util.List;

@Entity
@Table(name = "directores")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Director {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = true, unique = true)
    private String nombre;

    /**
     * Relación 1:N con Pelicula.
     * Director NO tiene la FK. 'mappedBy' dice: "La FK está en el campo 'director' de
     Pelicula".
     */
    @OneToMany(mappedBy = "director")
    @JsonIgnoreProperties("director")
    private List<Pelicula> peliculas;
}

```

F. Entidades Satélite (CATEGORIA, IDIOMA, PLATAFORMA)

Estas tres son muy similares. Son el lado inverso de la relación N:M que definimos en [Pelicula](#).

CATEGORIA:

```

Java
package com.dam2.Practica1.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

```

```

@Entity
@Table(name="categorias")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Categoria {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = true, length = 120, unique = true)
    private String nombre;

    /**
     * Relación N:M inversa con Pelicula.
     * No definimos @JoinTable aquí porque ya está en Pelicula.
     */
    @ManyToMany(mappedBy = "categorias")
    @JsonIgnoreProperties("categorias")
    private List<Pelicula> peliculas = new ArrayList<>();
}

```

IDIOMA:

```

Java
package com.dam2.Practica1.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "idiomas")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Idioma {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = true, length = 120, unique = true)
    private String nombre;

```

```

    @ManyToMany(mappedBy = "idiomas")
    @JsonIgnoreProperties("idiomas")
    private List<Película> películas = new ArrayList<>();
}

```

PLATAFORMA:

```

Java
package com.dam2.Practica1.models;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name="plataformas")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Plataforma {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = true, length = 120)
    private String nombre;

    @Column(nullable = true, length = 255)
    private String url;

    @ManyToMany(mappedBy = "plataformas")
    @JsonIgnoreProperties("plataformas")
    private List<Película> películas = new ArrayList<>();
}

```

3. Resumen de Claves y Lógica

¿Por qué he puesto las claves donde están?

1. Regla de Oro del 1:N (Uno a Muchos):

- En Base de Datos, la clave foránea (Foreign Key) **SIEMPRE** va en la tabla del lado "Muchos" (N).
- *Ejemplo Director-Película:* Un director tiene muchas pelis. Si pusieras la ID de la película en la tabla Director, solo podrías guardar una peli por director. Por eso, la tabla `peliculas` lleva la columna `director_id`.
- En Java: `@ManyToOne` lleva el `@JoinColumn`. `@OneToMany` lleva el `mappedBy`.

2. Regla de Oro del N:M (Muchos a Muchos):

- En Base de Datos, **SIEMPRE** hace falta una tercera tabla intermedia.
- En Java: Debes elegir un "Jefe" (Owner). No importa técnicamente cuál elijas, pero sí lógicamente.
- *Ejemplo Película-Categoría:* He puesto el `@JoinTable` en `Película`. ¿Por qué? Porque cuando creas una película en el sistema, dices "Es de Terror y Comedia". Es más natural gestionar las categorías desde la película que abrir la entidad "Terror" y añadirle todas las películas de la historia.

3. Gestión del JSON (Bucle Infinito):

- Si tienes `Película` -> lista de `Actores` -> lista de `Películas` -> lista de `Actores`... tu API explotará (`StackOverflowError`).
- He usado `@JsonIgnoreProperties("nombre_campo_lista")` en las listas inversas para cortar el bucle. Alternativamente se usa `@JsonManagedReference` (lado padre) y `@JsonBackReference` (lado hijo), pero `@JsonIgnoreProperties` es a menudo más flexible en relaciones N:M.